KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

제어문 1
조건문 if

조건에 따라
다른 줄을 실행하기

```
if money >= price:
    print("주문할 수 있어요")
elif total >= 10000:
    print("5% 할인")
else:
    print("할인 없음")
```

오늘의 순서

1 참과 거짓으로 길 고르기

왜 필요한가, 갈림길 그림, 가장 짧은 if, 조건은 True와 False

4–7쪽

2 if, else, elif

False면 건너뛰기, 들여쓰기 범위, else, elif, 순서, if 여러 개

8–13쪽

3 조건 합치기

and, if 안의 if, or, not in, 범위, 빈 값은 False

14–20쪽

4 for와 함께, 줄여 쓰기

골라 담기, 세기, items(), 한 줄 if else, get과 16 코드 다시 읽기

21–26쪽

5 자주 나는 오류

= 와 ==, 콜론과 들여쓰기, or 함정, 글자와 숫자 비교

27–30쪽

6 실습

장바구니 할인, 성적 등급, 메뉴판 없는 주문 빼기, 모양 고르기

31–34쪽

3쪽은 노트북 준비, 35–36쪽은 막힐 때와 한 장 요약이에요.

들어가기

16에서 이어서 해요

- ✓ hello-uv 노트북, 커널 .venv — 09 · 25–27쪽
- ✓ 비교 ==, >=, and, or, not — 12 · 3파트
- ✓ 리스트, for, f-string — 13
- ✓ in과 not in — 14
- ✓ 딕셔너리 menu와 get — 15
- ✓ 집합과 discard — 16

기억이 안 나면 오른쪽에 적힌 챕터를 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

1. 참과 거짓으로 길 고르기

지금까지는 적은 줄이 모두 실행됐어요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
order = "tea"
print(menu[order])
```

```
KeyError: 'tea'
```

메뉴판에 없는 주문이 오면 멈춰요

셀은 위에서 아래로 한 줄씩 모두 실행해요. tea가 없어도 print(menu[order])를 그대로 실행해서 KeyError가 나요.

있을 때만 꺼내고 싶어요

"tea가 메뉴판에 있으면 가격을 찍고, 없으면 없다고 말해요"처럼 조건에 따라 다른 줄을 실행해야 해요.

이번 챕터에서 이 if를 직접 써요.

15, 16에서 읽기만 한 코드

15 · get 비교

```
menu["cakee"] if "cakee" in menu else 5000
```

16 · discard 긴 코드

```
if "juice" in menus:
    menus.remove("juice")
```

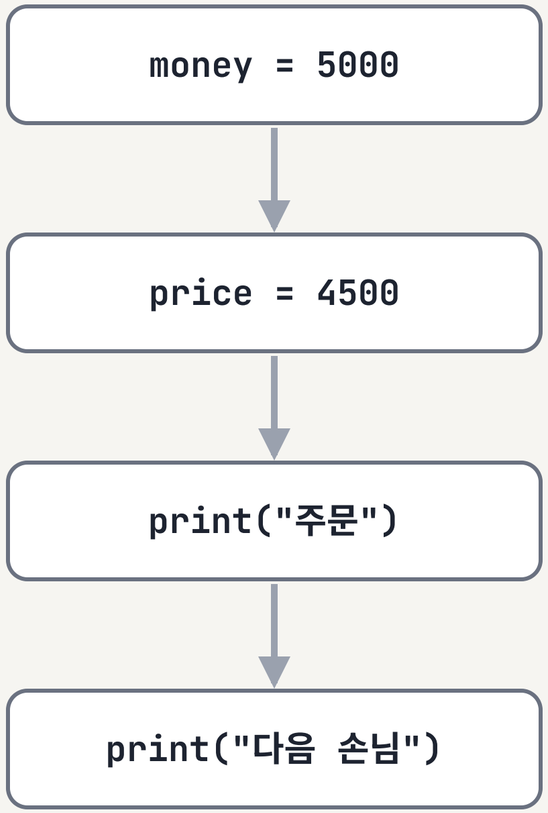
16 · 중복 없애기 긴 코드

```
for o in orders:
    if o not in kinds:
        kinds.append(o)
```

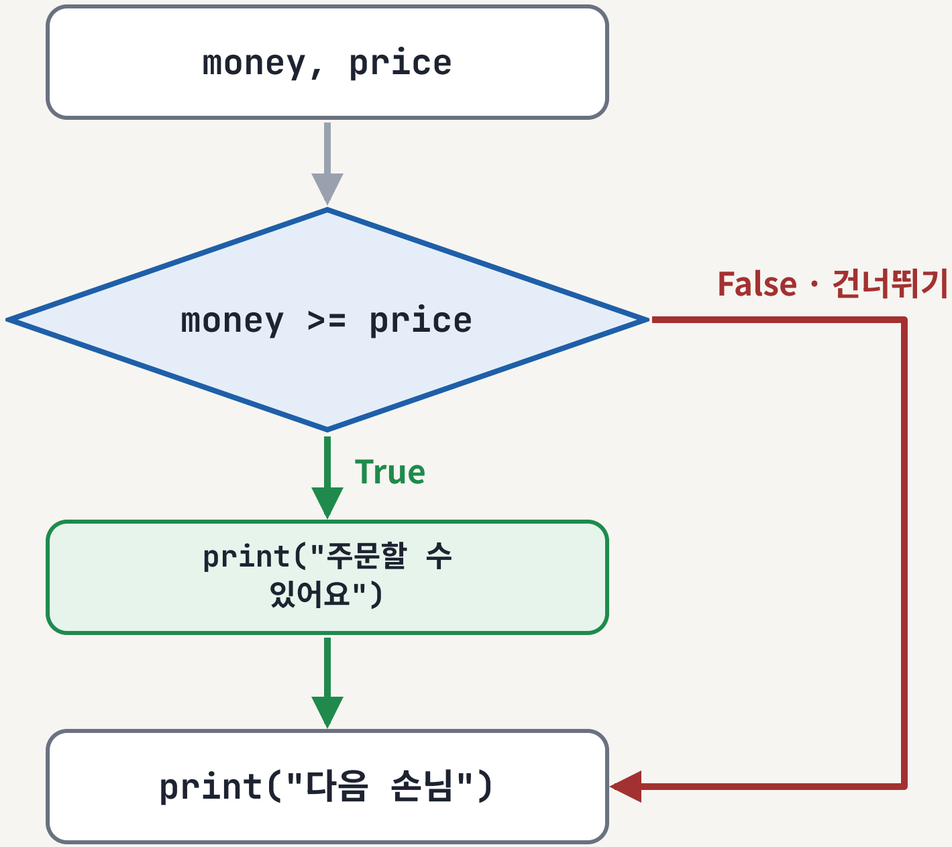
1. 참과 거짓으로 길 고르기

if는 코드에 갈림길을 만들어요

지금까지 · 한 줄로 쭉



if를 쓰면 · 조건으로 길을 골라요



갈림길은 마름모예요. True면 아래로, False면 옆길로 가요.

1. 참과 거짓으로 길 고르기

가장 짧은 if는 두 줄이에요

[2]
```python
money = 5000
price = 4500
if money >= price:
    print("주문할 수 있어요")
print("다음 손님")
```
조건 / True라서 실행 / if 밖 / Python

```
주문할 수 있어요
다음 손님
```

들여쓴 줄만 if에 속해요

들여쓴 print는 조건이 True일 때만 실행해요. 들여쓰지 않은 print("다음 손님")은 if와 상관없이 늘 실행해요.

`if` 만약

`money >= price` 조건 · True나 False가 나와요

`:` 콜론 · 여기까지가 조건

`␣␣␣␣` 4칸 들여쓰기 · if 안의 줄

들여쓰기는 Tab 한 번

VS Code는 Tab을 누르면 4칸을 넣어 줘요. 콜론 뒤 Enter를 치면 알아서 들여써요.

if 조건: 다음 줄을 들여쓰면 조건이 True일 때만 실행해요.

1. 참과 거짓으로 길 고르기

조건 자리에는 True나 False가 나오는 식을 적어요

[3]
```
print(money >= price)
print(money == price)
print("tea" in menu)
print("tea" not in menu)
print(type(money >= price))
```
Python

```
True
False
False
True
<class 'bool'>
```

bool은 참, 거짓 두 가지 값

비교와 in의 결과는 True 아니면 False예요. type으로 보면 bool이라고 나와요.

`==`	같아요	12에서 배움
`!=`	달라요	12에서 배움
`<` `>`	작아요, 커요	12에서 배움
`<=` `>=`	작거나 같아요, 크거나 같아요	12에서 배움
`in`	들어 있어요	14에서 배움
`not in`	안 들어 있어요	14에서 배움

if는 이 True와 False를 보고 길을 골라요.

2. if, else, elif

조건이 False면 들여쓴 줄을 건너뛰어요

[1]
```python
price = 4500
money = 5000
if money >= price:
    print("주문할 수 있어요")
print("다음 손님")
```
실행

Python

```
주문할 수 있어요
다음 손님
```

5000 >= 4500 → True

[2]
```python
money = 3000
if money >= price:
    print("주문할 수 있어요")
print("다음 손님")
```
건너뜀

Python

```
다음 손님
```

3000 >= 4500 → False

오류가 아니에요

False면 들여쓴 줄을 그냥 건너뛰고 if 다음 줄로 가요. 오른쪽 셀은 출력이 한 줄뿐이지만 정상이에요. price는 셀 [1]에서 넣은 4500이 그대로 남아 있어요.

같은 코드라도 값에 따라 실행하는 줄이 달라져요.

2. if, else, elif

들여쓴 줄까지가 if에 속해요

[3]
```
money = 3000
if money >= price:
    print("주문할 수 있어요")
    print("커피 나왔어요")
print("다음 손님")
```
if 안 · 둘 다 건너뜀

Python

```
다음 손님
```

[4]
```
money = 3000
if money >= price:
    print("주문할 수 있어요")
print("커피 나왔어요")
print("다음 손님")
```
if 안 · 건너뜀

if 밖 · 늘 실행

Python

```
커피 나왔어요
다음 손님
```

왼쪽 · 두 줄 모두 들여씀

돈이 모자라면 두 줄을 모두 건너뛰어요. 커피도 안 나와요.

오른쪽 · 한 줄만 들여씀

커피 나왔어요가 if 밖으로 나갔어요. 돈이 모자라도 커피가 나와요. 오류는 안 나지만 틀린 결과예요.

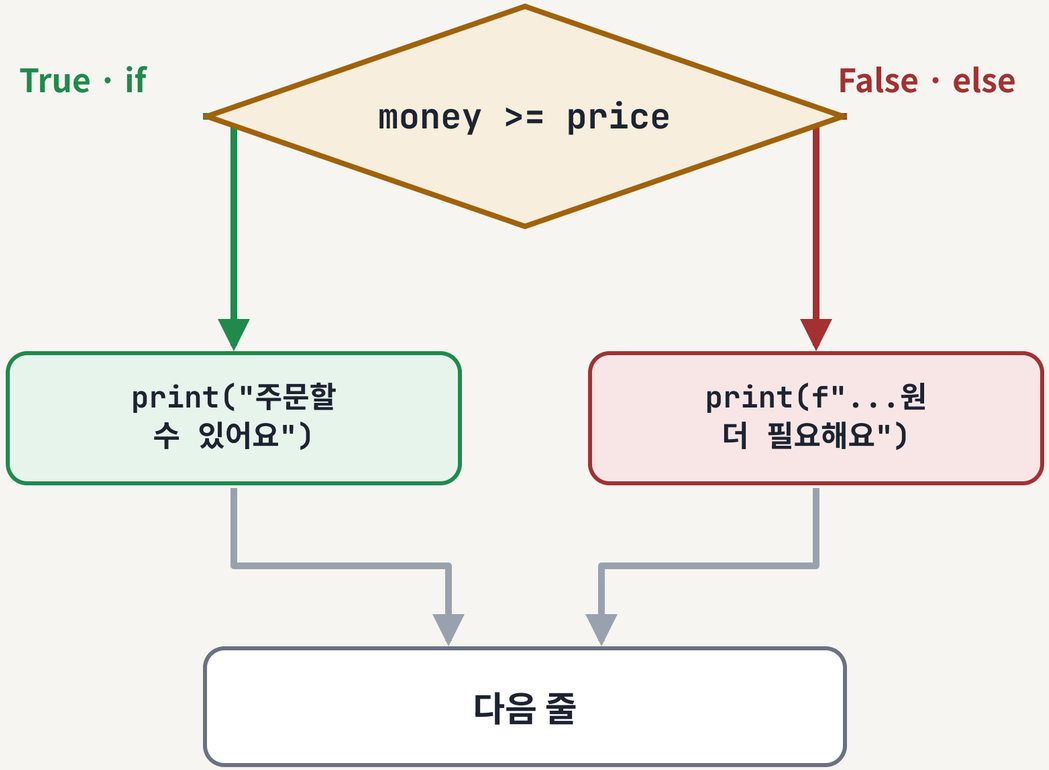
들여쓰기 4칸이 "어디까지 if인지"를 정해요.

2. if, else, elif

else는 조건이 False일 때 갈 길이에요

```
[5] money = 3000
    if money >= price:
        print("주문할 수 있어요")          건너뜀
    else:
        print(f"{price - money}원 더 필요해요")  실행
```
Python

```
1500원 더 필요해요
```



둘 중 하나는 꼭 실행해요

else 뒤에는 조건이 없고 콜론만 붙여요. if와 같은 칸에 맞춰 적고, 그 안은 다시 4칸 들여써요.

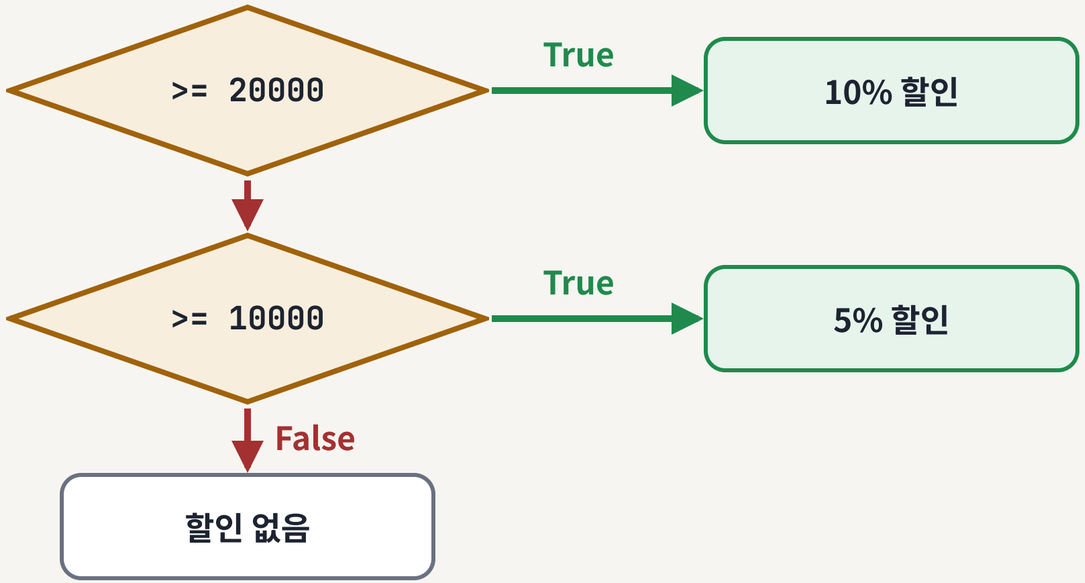
if는 True일 때, else는 False일 때. 두 길 중 하나만 가요.

2. if, else, elif

elif로 갈림길을 여러 개 만들어요

[6]
```python
total = 12000
if total >= 20000:                    # False
    print("10% 할인")
elif total >= 10000:                  # True · 실행
    print("5% 할인")
else:                                 # 안 봄
    print("할인 없음")
```

```
5% 할인
```



위에서부터 처음 True인 하나만

elif는 else if를 줄인 말이에요. 앞 조건이 False일 때만 다음 조건을 물어요. True를 만나면 그 길만 가고 나머지는 보지 않아요.

`total = 25000`	10% 할인
`total = 12000`	5% 할인
`total = 3000`	할인 없음

if 하나, elif 여러 개, else 하나. 그중 하나만 실행해요.

2. if, else, elif

조건 순서를 바꾸면 결과가 달라져요

[9]
```python
total = 25000
if total >= 10000:
    print("5% 할인")
elif total >= 20000:
    print("10% 할인")
else:
    print("할인 없음")
```

먼저 True

물어보지도 않음

Python

```
5% 할인
```

⚠ 25000원인데 5% 할인이 나왔어요

25000은 10000 이상이기도 해서 첫 줄에서 True가 나와요. 그 뒤의 elif는 보지 않아요.

좁은 조건부터 적어요

```python
if total >= 20000:      # 좁은 조건
elif total >= 10000:    # 넓은 조건
else:
```

앞 장의 순서예요. 25000을 넣으면 10% 할인이 나와요.

elif는 위에서부터 묻고 처음 True에서 멈춰요. 큰 금액 조건을 위에 둬요.

2. if, else, elif

if를 따로 쓰면 둘 다 실행될 수 있어요

[10]
```python
total = 25000
if total >= 20000:
    print("10% 할인")          # 실행
if total >= 10000:
    print("5% 할인")           # 또 실행
```

```
10% 할인
5% 할인
```

if 두 개 · 할인을 두 번 받아요

if마다 따로 물어요

if를 두 번 쓰면 갈림길이 두 개예요. 앞에서 True였어도 아래 if를 다시 물어서 둘 다 실행해요.

[7]
```python
total = 25000
if total >= 20000:
    print("10% 할인")
elif total >= 10000:
    print("5% 할인")
else:
    print("할인 없음")
```

```
10% 할인
```

if, elif · 하나만 받아요

하나만 고를 땐 elif

따로 따로 확인할 일이면 if를 여러 번 써요.

둘 중 하나만이면 elif, 각각 따로 확인하면 if 여러 개예요.

3. 조건 합치기

and는 두 조건이 모두 True일 때만 True예요

[1]
```python
height = 130
age = 9
if height >= 120 and age >= 7:
    print("탈 수 있어요")
else:
    print("다음에 타요")
```

탈 수 있어요

키 `130 >= 120` → `True`

나이 `9 >= 7` → `True`

and `True and True` → `True`

[2]
```python
height = 110
if height >= 120 and age >= 7:
    print("탈 수 있어요")
else:
    print("다음에 타요")
```

다음에 타요

키 `110 >= 120` → `False`

나이 `9 >= 7` → `True`

and `False and True` → `False`

12의 놀이기구 조건이에요. 이제 결과로 실행할 줄을 골라요.

3. 조건 합치기

if 안의 if는 and 한 줄로 줄일 수 있어요

[3]
```python
height = 130
if height >= 120:
    if age >= 7:          # 4칸
        print("탈 수 있어요")  # 8칸
```

탈 수 있어요

if 안에도 if를 넣을 수 있어요

바깥 if가 True일 때만 안쪽 if를 물어요. 안쪽 if 안의 줄은 8칸 들여써요. 두 조건이 모두 맞아야 실행하니 and와 같아요.

and로 줄이면 셀 [1]과 같은 결과

```python
if height >= 120 and age >= 7:
    print("탈 수 있어요")
```

탈 수 있어요

언제 안쪽 if를 쓰나요

바깥 조건이 맞은 뒤에 따로 할 일이 있을 때예요. 예를 들어 메뉴판에 있으면 먼저 가격을 찍고, 그다음 5000원 이하인지 다시 물어요. 할 일이 한 군데뿐이면 and가 더 읽기 쉬워요.

들여쓰기가 깊어지면 and로 합칠 수 있는지 먼저 봐요.

3. 조건 합치기

or는 하나만 True여도 True예요

```python
[4] is_member = False
total = 60000
if is_member or total >= 50000:
    print("할인해 드려요")
```

Python

할인해 드려요

회원 `is_member` → `False`

금액 `60000 >= 50000` → `True`

or `False or True` → `True`

and와 or 한눈에

앞	뒤	and	or
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

and는 모두, or는 하나라도

and는 하나라도 False면 False, or는 하나라도 True면 True예요.

회원이 아니어도 5만 원 이상이면 할인해요. 12의 할인 조건이에요.

3. 조건 합치기

not과 not in으로 반대를 물어요

[5]
```
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
order = "tea"
if not order in menu:                    not (in)
    print("메뉴판에 없어요")
if order not in menu:                    not in · 이쪽을 써요
    print("메뉴판에 없어요")
```
Python

메뉴판에 없어요

메뉴판에 없어요

in `"tea" in menu` → `False`

not `not False` → `True`

not in이 읽기 쉬워요

결과는 같아요. 영어 문장처럼 order not in menu로 읽혀서 실제로는 not in을 많이 써요.

없을 때 할 일만 있으면

"없으면 알려 주기"처럼 False일 때만 할 일이 있으면 else 대신 조건을 뒤집어요. 1파트 처음의 KeyError를 막는 첫 방법이에요.

not은 12에서 배웠어요

True와 False를 뒤집어요. not True는 False예요.

"없으면"은 not in, "아니면"은 !=로 적어요.

3. 조건 합치기

범위는 9 <= hour < 18처럼 이어 적어요

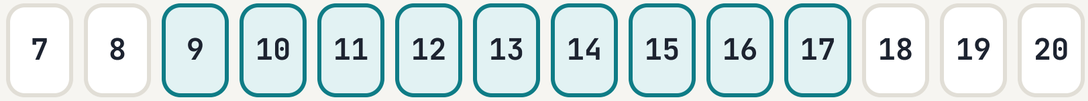
[6]
```python
hour = 14
if 9 <= hour < 18:
    print("영업 중이에요")
if hour >= 9 and hour < 18:
    print("영업 중이에요")
```

```
영업 중이에요
영업 중이에요
```

[7]
```python
hour = 20
if 9 <= hour < 18:
    print("영업 중이에요")
else:
    print("문 닫았어요")
```

```
문 닫았어요
```

영업시간 · 9시부터 18시 전까지



9는 들어가요 (<=)

18은 빠져요 (<)

같은 뜻이에요

`9 <= hour < 18` 줄인 방법

`hour >= 9 and hour < 18` 원래 방법

수학 시간의 부등식처럼

가운데에 변수를 두고 양쪽에 경계를 적어요.

"사이에 있나요?"는 작은 값 <= 변수 < 큰 값으로 물어요.

3. 조건 합치기

비어 있는 값은 False로 봐요

[8]
```
print(bool(0), bool(4500))
print(bool(""), bool("tea"))
print(bool([]), bool(["tea"]))
print(bool({}), bool(set()))
print(bool(None))
```
Python

```
False True
False True
False True
False False
False
```

bool()로 확인해요

bool(값)은 그 값을 if에 넣었을 때 True로 볼지 False로 볼지 알려 줘요. 0과 빈 것, None만 False예요.

종류	False로 봐요	True로 봐요
숫자	`0`	`4500`
문자열	`""`	`"tea"`
리스트	`[]`	`["tea"]`
딕셔너리, 집합	`{}` `set()`	`{"a": 1}`
15의 None	`None`	

0, 빈 문자열, 빈 리스트, 빈 딕셔너리, 빈 집합, None은 모두 False예요.

3. 조건 합치기

if orders:는 비어 있지 않은지 묻는 거예요

[9]
```python
orders = []
if orders:                                  # 줄인 방법
    print(len(orders), "잔 주문")
else:
    print("아직 주문이 없어요")
if len(orders) > 0:                         # 원래 방법
    print(len(orders), "잔 주문")
else:
    print("아직 주문이 없어요")
```

Python

```
아직 주문이 없어요
아직 주문이 없어요
```

같은 뜻이에요

`if orders:` 줄인 방법

`if len(orders) > 0:` 원래 방법

처음엔 원래 방법도 괜찮아요

if orders:는 "주문이 하나라도 있으면"이라고 읽어요. 헷갈리면 len(orders) > 0으로 적어도 결과가 같아요. 남이 쓴 코드에서 if orders:를 만나면 이 뜻으로 읽어요.

빈 리스트는 False라서 if 리스트:는 "하나라도 있으면"이에요.

4. for와 함께, 줄여 쓰기

for 안의 if로 골라 담아요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
prices = [4500, 5000, 6500, 4000]
cheap = []
for p in prices:
    if p <= 5000:                    # 4칸
        cheap.append(p)              # 8칸
print(cheap)
```

```
[4500, 5000, 4000]
```

한 바퀴 돌 때마다 if를 한 번 물어요

p가 바뀔 때마다 5000 이하인지 물어요. True인 값만 append해요. 들여쓰기가 for 4칸, if 8칸으로 두 번 들어가요.



빈 리스트 + for + if + append가 "골라 담기"의 기본 모양이에요.

4. for와 함께, 줄여 쓰기

조건에 맞는 것만 세요

[2]
```python
orders = ["coffee", "latte", "coffee", "cake", "tea", "coffee"]
n = 0
for o in orders:
    if o == "coffee":
        n += 1                                  # True일 때만 1 더하기
print(n)
print(orders.count("coffee"))
```

```
3
3
```

o	== "coffee"	n
coffee	True	1
latte	False	1
coffee	True	2
cake	False	2
tea	False	2
coffee	True	3

값 하나를 셀 땐 count가 짧아요

같은 값의 개수는 13의 count로 바로 세요. 5000원 이상처럼 조건이 붙으면 count로는 못 세서 for와 if를 써요.

세기는 n = 0에서 시작해 조건이 True일 때 n += 1이에요.

4. for와 함께, 줄여 쓰기

딕셔너리에서 조건에 맞는 메뉴를 찾아요

[3]
```python
for name, price in menu.items():
    if price <= 5000:
        print(name, price)
```

```
coffee 4500
latte 5000
```

coffee	latte	cake
4500	5000	6500
<= 5000 True	<= 5000 True	<= 5000 False

15의 items()에 if를 붙였어요

items()는 키와 값을 한 쌍씩 꺼내요. 값인 price로 조건을 묻고, 맞는 쌍만 찍어요. 키로 묻고 싶으면 if name == "cake":처럼 name을 써요.

메뉴판에 쓰면

5000원 이하 메뉴 추천, 품절인 메뉴 찾기, 이름에 te가 들어간 메뉴 찾기("te" in name)처럼 쓸 수 있어요.

for k, v in d.items(): 안에서 if로 키나 값을 물어요.

4. for와 함께, 줄여 쓰기

if와 else를 한 줄로 줄여 쓸 수 있어요

[4]

```python
total = 12000
if total >= 10000:                                  # 원래 방법 · 4줄
    msg = "5% 할인"
else:
    msg = "할인 없음"
print(msg)
msg = "5% 할인" if total >= 10000 else "할인 없음"      # 줄인 방법
print(msg)
```

```
5% 할인
5% 할인
```

값 하나를 고를 때만 줄여요

변수에 넣을 값이 둘 중 하나일 때 써요. 실행할 줄이 여러 개이거나 elif가 필요하면 원래 모양으로 적어요.

True일 때 값

`"5% 할인"`

조건

`if total >= 10000`

False일 때 값

`else "할인 없음"`

읽는 순서

"1만 원 이상이면 5% 할인, 아니면 할인 없음"

값1 if 조건 else 값2는 "조건이 True면 값1, 아니면 값2"예요.

4. for와 함께, 줄여 쓰기

15의 get을 이제 풀어서 읽을 수 있어요

[5]

```python
if "cakee" in menu:                                    # 1 · if, else
    price = menu["cakee"]
else:
    price = 5000
print(price)
print(menu["cakee"] if "cakee" in menu else 5000)
print(menu.get("cakee", 5000))                         # 3 · get
```

```
5000
5000
5000
```

get은 if를 대신 써 준 거예요

키가 있으면 그 값, 없으면 기본값. 세 줄 모두 같은 일을 해요. 짧을수록 실수할 곳이 줄어서 실제로는 get을 써요.

1 **if, else 4줄**
이번 챕터에서 배운 원래 모양

2 **한 줄 if else**
15의 참고 칸에서 읽기만 한 코드

3 **menu.get("cakee", 5000)**
15에서 배운 줄인 메서드

15에서는 왜 읽기만 했나요

if를 아직 몰라서예요. 이제 1번을 직접 칠 수 있어요.

줄인 메서드 안에는 이런 if가 들어 있어요. 원래 모양을 알면 결과를 읽기 쉬워요.

4. for와 함께, 줄여 쓰기

16에서 읽기만 한 코드를 직접 쳐 봐요

[6]
```python
menus = {"coffee", "cake"}
if "juice" in menus:                    # 원래 방법
    menus.remove("juice")
print(sorted(menus))
menus.discard("juice")                  # 줄인 방법
print(sorted(menus))
```
Python

```
['cake', 'coffee']
['cake', 'coffee']
```

discard = 있으면 remove

있는지 먼저 묻고 있을 때만 remove해요. juice는 없어서 아무 일도 없어요. discard 한 줄과 결과가 같아요.

[7]
```python
kinds = []                              # 원래 방법
for o in orders:
    if o not in kinds:
        kinds.append(o)
print(kinds)
print(sorted(set(orders)))              # 줄인 방법
```
Python

```
['coffee', 'latte', 'cake', 'tea']
['cake', 'coffee', 'latte', 'tea']
```

set() = 없을 때만 append

리스트는 처음 나온 순서를 지키고, 집합은 sorted()로 줄 세워 찍었어요. 담긴 값은 같아요.

16의 "참고, 읽기만" 칸은 이제 모두 직접 칠 수 있어요.

5. 자주 나는 오류

if 뒤에는 = 하나가 아니라 == 두 개예요

[1]
```
order = "coffee"
if order = "coffee":                                    line 2
    print("커피예요")
```
Python

```
Cell In[1], line 2
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

오류 메시지 읽기

line 2는 둘째 줄이 문제라는 뜻이에요.

Maybe you meant '==' 는 "== 를 쓰려던 거 아니에요?"라고 고칠 곳을 알려 주는 말이에요.

:= 는 배우지 않은 기호라 신경 쓰지 않아요.

12에서 본 규칙 그대로

`order = "coffee"` 넣기

`if order == "coffee":` 묻기

SyntaxError는 실행하기 전에 문법을 검사하다 나서, 셀의 어느 줄도 실행되지 않아요.

SyntaxError가 나면 line 숫자의 줄부터 봐요.

5. 자주 나는 오류

콜론과 들여쓰기를 빠뜨리면 실행 전에 멈춰요

[2]
```python
if order == "coffee"
    print("커피예요")
```

```
SyntaxError: expected ':'
```

콜론이 없어요

`if 줄 끝에 : 붙이기`

[3]
```python
if order == "coffee":
print("커피예요")
```

```
IndentationError: expected an indented block after 'if' statement on line 1
```

if 안에 들여쓴 줄이 없어요

`print 앞에 4칸`

[4]
```python
if order == "coffee":
    print("커피예요")
      print("나왔어요")
```

```
IndentationError: unexpected indent
```

들여쓰기 칸 수가 달라요

`같은 칸(4칸)에 맞추기`

IndentationError는 들여쓰기 오류예요. 칸 수를 4칸에 맞춰요.

5. 자주 나는 오류

or 뒤에도 비교를 다시 적어요

[5]
```python
order = "cake"
if order == "coffee" or "latte":          # 늘 True
    print("커피 종류예요")
print(order == "coffee" or "latte")
```

```
커피 종류예요
latte
```

[6]
```python
if order == "coffee" or order == "latte":
    print("커피 종류예요")
else:
    print("커피 종류가 아니에요")
print(order in ["coffee", "latte"])
```

```
커피 종류가 아니에요
False
```

앞 `order == "coffee"` → `False`

or `False or "latte"`

결과 `"latte"`

"latte"는 빈 문자열이 아니라 True로 봐요

3파트에서 본 규칙이에요. or 뒤가 비교가 아니라 글자 하나라서 늘 True로 봐요.

고치는 두 가지

비교를 둘 다 적거나, in과 리스트로 물어요.

말로는 "coffee 또는 latte"지만 코드는 order == 를 두 번 적어요.

5. 자주 나는 오류

글자와 숫자는 크기를 비교할 수 없어요

[7]
```python
price = "4500"
if price >= 4000:
    print("4000원 넘어요")
```

```
TypeError: '>=' not supported between instances of 'str' and 'int'
```

[8]
```python
if int(price) >= 4000:
    print("4000원 넘어요")
```

```
4000원 넘어요
```

price

"4500"

따옴표가 있으면 글자예요

"4500"은 str, 4000은 int라서 크고 작음을 비교할 수 없어요. str과 int 사이에서는 >=를 지원하지 않는다는 뜻이에요.

12의 int()로 바꿔요

int(price)는 숫자 4500이 돼요. ==는 오류 없이 False가 나와서 더 찾기 어려워요.

비교 오류가 나면 type()으로 두 값의 종류부터 봐요.

6. 실습

장바구니 금액에 따라 할인해요

[1]

```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
cart = {"coffee": 2, "cake": 1}
total = 0
for name, qty in cart.items():                    # 15 · 합계
    total += menu[name] * qty
if total >= 20000:                                # elif · 할인율
    rate = 10
elif total >= 10000:
    rate = 5
else:
    rate = 0
sale = total * rate // 100                        # 12 · 몫
print(f"합계 {total:>7,}원")
print(f"할인 {sale:>7,}원 ({rate}%)")
print(f"결제 {total - sale:>7,}원")
```

Python

```
합 계   15,500원
할 인      775원 (5%)
결 제   14,725원
```

coffee 2잔
4,500 × 2 = 9,000

cake 1개
6,500 × 1 = 6,500

합계 15,500
10000 이상 · 5%

할인
15500 × 5 // 100 = 775

for로 합계를 내고, elif로 할인율 하나를 골라요.

6. 실습

점수로 등급을 매겨요

[2]

```
scores = [["kim", 92], ["lee", 78], ["park", 85], ["choi", 59]]
for name, score in scores:
    if score >= 90:
        grade = "A"
    elif score >= 80:
        grade = "B"
    elif score >= 70:
        grade = "C"
    else:
        grade = "F"
    print(f"{name:<5}{score:>3}  {grade}")
```

for 안 · 4칸 Python

```
kim   92  A
lee   78  C
park  85  B
choi  59  F
```

A 90 이상

B 80 이상

C 70 이상

F 나머지

계단은 위에서부터

높은 점수 조건을 먼저 적어요. 2파트의 순서 규칙이에요.

for 안에 elif 계단을 넣으면 한 명씩 등급이 나와요.

6. 실습

메뉴판에 없는 주문은 빼고 계산해요

[3]

```python
orders = ["coffee", "tea", "cake", "coffee", "juice"]
total = 0
skipped = []
for o in orders:
    if o in menu:                                  # 있으면 계산
        print(f"{o:<8}{menu[o]:>6,}원")
        total += menu[o]
    else:                                          # 없으면 따로 모으기
        skipped.append(o)
print(f"합계 {total:,}원")
if skipped:                                        # if 리스트:
    print("준비 중:", skipped)
```

Python

```
coffee    4,500원
cake      6,500원
coffee    4,500원
합계 15,500원
준비 중: ['tea', 'juice']
```

KeyError가 안 나요

1파트 처음에 멈췄던 tea를 이제 if로 먼저 물어서 건너뛰어요.

빠진 주문도 알려 줘요

else에서 skipped에 모았다가 끝에 한 번 찍어요. 하나도 없으면 3파트의 규칙대로 찍지 않아요.

16과 비교

16은 & 로 먼저 걸렀어요. 이번엔 for 안에서 하나씩 물어요.

if는 KeyError가 날 줄 앞에서 먼저 물어 막아 줘요.

6. 실습

어떤 모양을 쓸지 이렇게 골라요

`if`	**할 일이 한 가지, 조건이 맞을 때만**	예: 있으면 알려 주기
`if, else`	**둘 중 하나는 꼭**	예: 살 수 있음, 돈이 모자람
`if, elif, else`	**셋 이상 중 하나만**	예: 할인율, 등급
`if 여러 개`	**각각 따로 확인**	예: 쿠폰 여러 장
`and, or, not`	**조건 여러 개를 하나로**	예: 키와 나이, 회원이거나 5만 원
`값1 if 조건 else 값2`	**변수에 둘 중 하나 넣기**	예: msg 고르기

처음에는 길게 적고, 같은 모양이 반복되면 줄여 써요.

정리

막히면 오류의 마지막 줄을 봐요

1	`SyntaxError: invalid syntax. Maybe you meant '=='...` if 뒤에 = 하나를 썼어요	`==`
2	`SyntaxError: expected ':'` if, elif, else 줄 끝에 콜론이 없어요	끝에 :
3	`IndentationError: expected an indented block` if 다음 줄을 들여쓰지 않았어요	4칸 들여쓰기
4	`IndentationError: unexpected indent` 들여쓰기 칸 수가 서로 달라요	같은 칸에 맞추기
5	`TypeError: '>=' not supported ... 'str' and 'int'` 글자와 숫자를 비교했어요	`int(price)`
6	**조건이 늘 True예요** or 뒤에 비교 없이 값만 적었어요	`o == "a" or o == "b"`
7	**결과가 이상해요** elif 순서, 들여쓰기 범위를 봐요	좁은 조건부터

오류가 안 나도 결과가 이상하면 어느 줄이 실행됐는지 print로 찍어 봐요.

한 장 요약

`if 조건:`

True일 때만 실행

`else:`

False일 때 갈 길

`elif 조건:`

위에서부터 처음 True 하나

`    (4칸)`

들여쓴 줄까지 if 안

`a and b` `a or b`

둘 다, 하나라도

`x not in menu`

없으면

`9 <= h < 18`

범위 묻기

`if orders:`

비어 있지 않으면

조건 뒤에 콜론, 다음 줄은 4칸. 묻기는 ==, 넣기는 =

다음 챕터 18에서는 while로 조건이 True인 동안 반복하고, break로 멈춰요